Customer Financial Behaviour and Risk Analysis Report

Task 1

Data Cleaning and Standardization

- **Currency Handling:** All `TransactionAmount` entries were stripped of special characters (e.g., "£") and converted to float types for accurate calculation.
- **Standardization:** The `AccountType` and `TransactionType` fields were standardized to ensure consistent grouping for "Savings," "Current," and "Credit" accounts.
- **Validation:** `TransactionDate` was formatted to a datetime object, enabling the identification of monthly trends and transaction gaps.

Task 1

```
# Remove symbols like '£' or ',' and convert to numeric float
df['TransactionAmount'] = df['TransactionAmount'].replace(r'[£,]', '', regex=True).astype(float)
```

```
# Ensuring both amount and balance are float types for calculation
df['TransactionAmount'] = df['TransactionAmount'].astype(float)
df['AccountBalance'] = df['AccountBalance'].astype(float)
print("Currency fields converted to float.")
```

```
Currency fields converted to float.
```

```
# Converting TransactionDate to datetime objects
df['TransactionDate'] = pd.to_datetime(df['TransactionDate'])
print("Date column validated and formatted.")
```

```
Date column validated and formatted.
```

```
# Standardize strings to title case to avoid duplicates like 'savings' and 'Savings'
df['AccountType'] = df['AccountType'].str.strip().str.title()
df['TransactionType'] = df['TransactionType'].str.strip().str.title()
print("Account types and transaction categories standardized.")
```

```
Account types and transaction categories standardized.
```

Task 2

Transactional Analysis & Trends

- **Net Inflow:** By calculating the difference between credits (deposits) and debits (withdrawals/payments), we identified the highest and lowest performing accounts.
- **Dormancy Tracking:** Accounts were flagged as "Dormant" if they exhibited a gap of two months or more between consecutive transactions.
- **Trend Analysis:** Visualization of monthly credits vs. debits revealed seasonal peaks in transaction volume, aiding in liquidity planning.

Task 2

```python
# Create Year and Month columns
df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.month

# Summarize total credits (Deposits) and debits (Withdrawals/Payments)
summary = df.groupby(['Year', 'Month', 'TransactionType'])['TransactionAmount'].sum().unstack().fillna(0)
print(summary)
```

```python
# Aggregating data for plotting
trends = df.groupby(df['TransactionDate'].dt.to_period('M'))['TransactionAmount'].sum()
trends.plot(kind='line', marker='o', title='Transaction Trends Over Time')
plt.ylabel('Total Amount')
plt.show()
```

```python
# Net inflow = Deposits - (Withdrawals + Payments)
inflow = df.pivot_table(index='AccountID', columns='TransactionType', values='TransactionAmount', aggfunc='sum').fillna(0)
inflow['NetInflow'] = inflow.get('Deposit', 0) - (inflow.get('Withdrawal', 0) + inflow.get('Payment', 0))

top_accounts = inflow.sort_values(by='NetInflow', ascending=False).head(5)
bottom_accounts = inflow.sort_values(by='NetInflow', ascending=True).head(5)

print(top_accounts)
```

```python
# Accounts with a gap of 2 months or more
df = df.sort_values(['AccountID', 'TransactionDate'])
df['DateDiff'] = df.groupby('AccountID')['TransactionDate'].diff().dt.days

# Flagging accounts where the gap exceeds 60 days (approx 2 months)
dormant_accounts = df[df['DateDiff'] > 60]['AccountID'].unique()
print(f"Dormant Account IDs: {dormant_accounts}")
```

Task 3

Customer Profiling

- **Activity Levels:** Customers were categorized into **High**, **Medium**, and **Low** activity groups based on transaction frequency.
- **Segment Insights:** * **High-Net Inflow:** These represent the most stable and profitable customer base.
    - **High-Frequency/Low-Balance:** These accounts generate high operational costs with high overdraft risk.
    - **Near-Zero Balance:** Identified as high-risk for default or overdraft incidents.

Task 3

```python
# Calculating transaction frequency
activity_counts = df.groupby('AccountID')['TransactionID'].count()

# Applying the rubric for activity levels
def get_activity_level(count):
    if count > 20: return 'High Activity'
    elif count >= 10: return 'Medium Activity'
    else: return 'Low Activity'

df['ActivityLevel'] = df['AccountID'].map(activity_counts.apply(get_activity_level))
print(df['ActivityLevel'].value_counts())
```

```python
segmentation = df.groupby('AccountID').agg({
    'AccountBalance': 'mean',
    'TransactionID': 'count'
}).rename(columns={'TransactionID': 'Volume', 'AccountBalance': 'AvgBalance'})
print(segmentation)
```

```python
# Filtering accounts with high net inflow (top 10%)
threshold_inflow = inflow.quantile(0.9)
high_inflow_profiles = inflow[inflow > threshold_inflow]
print(f"Identified {len(high_inflow_profiles)} High-net inflow accounts.")
```

```python
# High frequency (> median volume) and Low balance (< median balance)
median_vol = customer_segments['TxnVolume'].median()
median_bal = customer_segments['AvgBalance'].median()

hf_lb_accounts = customer_segments[(customer_segments['TxnVolume'] > median_vol) & (customer_segments['AvgBalance'] < median_bal)]
print(f"High-frequency low-balance accounts: {len(hf_lb_accounts)}")
```

```python
# Flagging accounts with balance below £100
near_zero_profiles = customer_segments[customer_segments['AvgBalance'] < 100]
print(f"Accounts with near-zero balances: {len(near_zero_profiles)}")
```

Task 4

Financial Risk Identification

- **Volatility:** Balance volatility was calculated using standard deviation to identify accounts with unpredictable financial stability.
- **Anomaly Detection:** Statistical methods like **Z-scores** were applied to detect "outlier" transactions that deviate significantly from the norm.
- **Suspicious Behavior:** Accounts were highlighted for manual review if they combined high `RiskScore` values with irregular transaction patterns.

Task 4

```
# Criterion: Withdrawal > 4000 or Account Balance < 0
large_withdrawals = df[(df['TransactionType'] == 'Withdrawal') & (df['TransactionAmount'] > 4000)]
overdraft_incidents = df[df['AccountBalance'] < 0]

print(f"Large withdrawals: {len(large_withdrawals)}, Overdraft incidents: {len(overdraft_incidents)}")
```

```
# Measuring how much the account balance fluctuates
balance_volatility = df.groupby('AccountID')['AccountBalance'].std().fillna(0)
print("Balance Volatility (Top 5 volatile accounts):\n", balance_volatility.sort_values(ascending=False).head())
```

```
# Using Z-score to detect transaction amount anomalies (Threshold > 3)
df['Z_Score_Amount'] = stats.zscore(df['TransactionAmount'])
anomalies = df[df['Z_Score_Amount'].abs() > 3]
print(f"Statistical anomalies detected: {len(anomalies)}")
```

```
# Criteria for Suspicious Behavior: High Volatility + High Risk Score + Anomaly Flag
risk_threshold = df['RiskScore'].quantile(0.9) # Top 10% risk scores

suspicious_customers = df[
    (df['AccountID'].isin(anomalies['AccountID'])) & (df['RiskScore'] > risk_threshold)]['AccountID'].unique()

print(f"CUSTOMERS HIGHLIGHTED FOR INVESTIGATION: {list(suspicious_customers)}")
```

Task 5

Visualisation

- Conduct extensive exploratory data analysis with attractive visualizations for your findings

Task 5

```
plt.figure(figsize=(10,6))
sns.boxplot(x='AccountType', y='RiskScore', data=df)
plt.title('Risk Score Distribution by Account Type')
plt.show()

plt.figure(figsize=(10,6))
sns.scatterplot(x='Account Balance', y='TransactionAmount', hue='TransactionType', data=df)
plt.title('Balance vs. Transaction Amount')
plt.show()
```

Task 6

Statistical Hypothesis Testing

- **Volume vs. Balance:** We tested whether high-volume transaction accounts statistically maintain higher average balances.
- **Finding:** (Based on analysis) The results determine if transaction frequency is a reliable proxy for account wealth and stability.

Task 6

```
# Define high volume as > 10 transactions
high_vol_ids = segmentation[segmentation['Volume'] > 10].index
low_vol_ids = segmentation[segmentation['Volume'] <= 10].index

high_vol_balances = df[df['AccountID'].isin(high_vol_ids)]['AccountBalance']
low_vol_balances = df[df['AccountID'].isin(low_vol_ids)]['AccountBalance']

t_stat, p_val = stats.ttest_ind(high_vol_balances, low_vol_balances)
print(f"P-Value: {p_val}") # If p < 0.05, the difference is statistically significant
```

```
# Testing if Savings accounts have significantly different risk scores than Credit accounts
savings_risk = df[df['AccountType'] == 'Savings']['RiskScore']
credit_risk = df[df['AccountType'] == 'Credit']['RiskScore']

t_stat_risk, p_val_risk = stats.ttest_ind(savings_risk, credit_risk)
print(f"Risk Hypothesis P-Value: {p_val_risk:.4f}")
```

Recommendations for Decision Makers

1. **Retention:** Launch targeted engagement campaigns for accounts flagged as "Dormant" (2+ month gap) to prevent churn.

2. **Monitoring:** Implement real-time alerts for the "Suspicious Behavior" segment identified in Task 4 to mitigate potential fraud.
3. **Product Alignment:** Offer credit-building products to "High-frequency/Low-balance" segments to help stabilize their account performance.

Video : https://drive.google.com/file/d/13d_zguXGWL2S9j-xU1CRc6gOfeYSY5ka/view?usp=sharing